

AI Pioneers

CCA-F | Module 5

Lab 5.3

Trust & Traceability: **Human Review, Confidence & Provenance**

Scenario: An AI Compliance Reviewer for Quarterly Financial Reports

Module	M5 — Context Management & Reliability
Lab type	Hands-on · Self-paced or Instructor-Led (Optional) · Claude API (Python)
Duration	~45 minutes (hands-on)
Environment	Blue Labs VM · Python 3.10+ · anthropic + python-dotenv · ANTHROPIC_API_KEY
Sections	S5 (Human Review & Confidence Calibration) S6 (Information Provenance & Uncertainty Handling)

Lab Objectives

This lab is framed around one real-world use case — an AI compliance reviewer for quarterly financial reports — because the same trust techniques apply to any regulated AI workflow (loan underwriting, medical coding, insurance claims, contract review). By the end you will be able to:

- **Calibrate confidence for auto-clear vs. escalation** — give each finding a 0..1 score so high-confidence findings clear automatically and low-confidence ones route to a human reviewer, the same triage a bank uses to decide which alerts need sign-off.
- **Attach tamper-proof provenance** — cite the exact line and quoted text for every finding, attached locally from the real report (never copied by the model) so an auditor can verify each flag in seconds and the citation can't be hallucinated.
- **Distinguish confirmed vs. contested findings** — run two independent review passes and compare: lines both flag are confirmed, lines only one flags are contested and surfaced for human judgment — the “four-eyes” dual-review control regulators expect.

1. Overview

An AI finding is only useful in a regulated industry if a human can trust it. This lab builds a command-line compliance reviewer that scans a quarterly financial report and produces findings a compliance officer will act on — and makes each finding trustworthy three ways. First, every finding carries a confidence score, and a threshold routes high-confidence findings to auto-clear and the rest to human review. Second, every finding cites its exact source line, attached locally from the real report so the citation cannot be hallucinated. Third, the review runs twice with different prompts; agreement across passes confirms a finding, disagreement marks it contested for a human. You run `main.py` and read three buckets: `auto_clear`, `human_review`, and `contested`. This lab covers Module 5 sections S5 and S6 and is optional.

SN	Demo	What you will do	Core idea
Ex 1	S5 — Human Review & Confidence	Score each finding 0..1; route by a threshold to auto-clear or human review.	Confidence triages where human attention goes.
Ex 2	S6 — Provenance	Attach the exact source line + quote locally to every finding.	Cite from real data; never trust the model to copy it.
Ex 3	S5 + S6 — Confirmed vs Contested	Run two passes; confirm on agreement, contest on disagreement.	Corroboration earns trust; disagreement goes to a human.

2. Scenario

You are building an AI reviewer for quarterly financial reports. It scans a report for compliance issues — missing disclosures, risky language, numbers that don't add up — and produces findings that a human compliance officer will see. Because this is a regulated context, the output cannot be a black box: a wrong auto-approval is a missed compliance issue, and an unverifiable flag wastes a reviewer's time or, worse, gets trusted blindly.

Your job is to make the reviewer's findings trustworthy. Each finding must say how sure it is, point at exactly where in the report it came from, and be corroborated by a second independent pass before it is trusted without human eyes.

You will build a command-line reviewer that:

- Reads a sample financial report (plain text, one statement per line) and asks Claude for findings as structured JSON, each with a confidence score and a source line number.
- Attaches the exact quoted line to every finding from the real report data — so provenance can't be hallucinated — and runs the review twice with slightly different prompts.
- Splits the results into three buckets: `auto_clear` (confirmed, high confidence), `human_review` (low confidence), and `contested` (the two passes disagreed).

The finding & the three buckets

```
finding shape: {line, quote, flag, confidence}  (quote attached locally, not by
the model)
auto_clear    → in BOTH passes and avg confidence >= 0.75
human_review  → in both passes but below the confidence threshold
contested     → flagged by only one of the two passes
```

Why these three together?

They answer the three questions a reviewer asks of any AI finding: *how sure* (confidence → routing), *says who / from where* (provenance → the exact line), and *does anything corroborate it* (two passes → confirmed vs contested). Confidence without provenance is an unverifiable guess; provenance without corroboration is a single fallible opinion. Together they make findings calibrated, traceable, and corroborated — the bar a regulated workflow has to clear.

3. Pre-requisites

3.1 Skilljar Videos — Completed in Earlier Modules

Module 5 introduces no new Skilljar courses. Sections S5 and S6 have no dedicated lesson in any CCA-F course — they are built in this lab. The closest supporting material is below; the techniques themselves are hands-on here.

Course	Lessons to watch	Covers
(none — built in this lab)	Human review & confidence calibration	S5
(none — built in this lab)	Information provenance & uncertainty handling	S6
AI Capabilities and Limitations	Optional background · evaluation & provenance patterns (docs.claude.com)	S5, S6

This lab is optional and assumes you have completed Labs 5.1 and 5.2. Bring questions to the Doubt Session.

3.2 Environment Check

Run these checks before the session starts. If any step fails, contact the instructor or check the Blue Labs SOP.

- Start your Blue Labs VM: open <https://www.bluelabs.studio/>, locate your VM, click Start, then Connect.
- Unzip the lab bundle and open the folder in VS Code (File > Open Folder...):

```
unzip lab_5_3_trust_traceability.zip
code lab_5_3_trust_traceability
```

- Create a Python environment and install the dependencies:

```
python -m venv .venv && source .venv/bin/activate
# Windows: .venv\Scripts\Activate.ps1
pip install -r requirements.txt          # anthropic + python-dotenv
```

- Add your API key. On the fresh starter, python main.py stops at the first TODO with a clear message; once your TODOs are done it runs the reviewer end-to-end:

```
cp .env.example .env          # paste your key into .env
python main.py
```

Model, key & what runs offline

The reviewer reads the model from `MODEL_NAME` (default `claude-sonnet-4-5`) and the key from `ANTHROPIC_API_KEY`. Only the review passes call the API; the bucketing logic in `confidence.py` and the report data in `sample_report.py` are pure Python you can read and run without a key. Findings are parsed with `json.loads` (not `regex`), and a parse failure is treated as a low-confidence skip rather than a crash.

4. Demos

Work through these from the lab folder. Each demo has one TODO to complete (see each file's header); once your TODOs are done, run `main.py` to see all three buckets together.

Demo 1: Confidence calibration & routing (Human Review, S5) ~15 min

Background

A single finding with a confidence number is not enough on its own — you need a policy for what to *do* with that number. `confidence.py` applies a `CONFIDENCE_THRESHOLD` of 0.75: a confirmed finding at or above it auto-clears; below it, it routes to a human. That is the triage a regulated workflow uses to spend scarce reviewer attention only where the model is unsure.

Task

Implement the bucketing and routing in `confidence.py`, then tune the threshold and watch the buckets shift.

Step 1 — Read the threshold and routing

Open `confidence.py`. Confirmed findings (present in both passes) are averaged and compared to the threshold:

```
CONFIDENCE_THRESHOLD = 0.75
...
avg_conf = (a["confidence"] + b["confidence"]) / 2
if avg_conf >= CONFIDENCE_THRESHOLD:
    auto_clear.append(entry)
else:
    human_review.append(entry)
```

Step 2 — Run and read the buckets

```
python main.py
```

Note which findings land in AUTO-CLEAR vs HUMAN REVIEW — a confirmed finding above 0.75 clears, a confirmed-but-uncertain one is held for a person.

Step 3 — Tune the dial

Lower `CONFIDENCE_THRESHOLD` to 0.5 and re-run; more findings auto-clear. Raise it to 0.9; more route to human review. There is no universally right value — it trades throughput against the cost of a missed issue.

What good looks like

The same confirmed finding routes differently based on its averaged confidence — high-confidence clears automatically, lower-confidence is held for a human. The threshold is an explicit, tunable dial, and the routing logic lives in `confidence.py`, not scattered through `main`.

Reflection Questions

- Why route low-confidence findings to a human instead of just lowering the bar and auto-clearing everything?
- The threshold is 0.75. What happens to throughput and to risk as you raise or lower it?
- Why calibrate confidence per finding rather than reporting one aggregate accuracy for the whole report?

Demo 2: Provenance — cite the source line (Provenance, S6) ~15 min

Background

Every finding must be verifiable. The model returns only a *line number* and a flag; `reviewer.py` attaches the quote from the real report via `get_quote(line)` — the model never copies the text, so the citation can't be hallucinated. A finding whose line is out of range is dropped, and a malformed model response degrades to zero findings instead of crashing.

Task

Implement the tolerant parser and the local-quote attachment in `reviewer.py`, so a model finding becomes a provenance-backed finding whose quote always comes from the source data.

Step 1 — See where the quote comes from

In `reviewer.py`, the model's JSON has no quote text — the quote is attached locally from `sample_report.get_quote`:

```
line = f.get("line")
quote = get_quote(line)          # from OUR data, by line number
if not quote:
    continue                     # line out of range -> drop (unverifiable)
cleaned.append({"line": line, "quote": quote,
                "flag": str(f.get("flag", "unspecified")),
                "confidence": float(f.get("confidence", 0.0))})
```

Step 2 — Note the tolerant parser

The `_parse_json_array` helper strips ``` fences and, on a JSON error, returns `[]` rather than raising — a parse failure becomes a safe, low-confidence skip:

```
try:
    return json.loads(text)
except json.JSONDecodeError:
    return []                    # treat as zero findings, don't crash the run
```

Step 3 — Verify provenance in the output

Run `python main.py` and confirm every printed finding shows its line number and exact quote — that is the provenance a reviewer clicks to verify. Try editing a line in `sample_report.py`; the quote in the output changes to match, because it is read from the source.

What good looks like

Each finding carries the exact source line and quoted text, pulled from the real report rather than the model — an auditor can verify any flag in one click. An out-of-range line is dropped because it has no verifiable source, and a parse failure yields no findings instead of a crash.

Reflection Questions

- The model returns only a line number; the quote is attached locally from the report. Why not let the model return the quote text too?
- The parser treats a JSON parse failure as zero findings rather than crashing. Why is that the right call in this pipeline?
- A finding whose line number is out of range is dropped. Why validate the line against the real report?

Demo 3: Confirmed vs. contested via two passes (Confirmed vs Contested, S5 + S6) ~15 min

Background

Corroboration earns trust. `main.py` runs the review twice with different prompts — a `strict` reviewer and a `general` one — two semi-independent opinions. Lines both passes flag are **confirmed**; lines only one flags are **contested** and surfaced for human judgment. This is the “four-eyes” dual-review control regulators expect, implemented with models.

Task

Wire the two passes and the confirmed-vs-contested comparison (`main.py` and `confidence.py`), and see why disagreement is surfaced rather than resolved automatically.

Step 1 — Two prompts, two passes

In `reviewer.py` the two prompts (`PROMPT_STRICT` and `PROMPT_GENERAL`) give independent passes; `main.py` runs both:

```
pass_a = review(report_text, mode="strict")
pass_b = review(report_text, mode="general")
buckets = bucket(pass_a, pass_b)
```

Step 2 — How agreement is decided

In `confidence.py`, findings on the same line number are “the same finding.” Both passes → confirmed (then routed by confidence); one pass only → contested:

```
if a and b:                                # both passes flagged this line
    ... confirmed -> auto_clear or human_review by confidence
```

```
else:
    # only one pass flagged it
    contested.append({... "flag_pass_a": ..., "flag_pass_b": ...})
```

Step 3 — Read the contested bucket

Run `python main.py` and look at `CONTESTED` — each entry shows what each pass said (one may be `None`), so a human can adjudicate. These are exactly the ambiguous cases where judgment matters most.

What good looks like

Agreement plus confidence decides the bucket: confirmed and high-confidence → auto-clear; confirmed but uncertain → human review; seen by only one pass → contested, with both passes' flags shown. Confirmed-but-unsure never auto-clears, and disagreement is surfaced, not silently resolved.

Reflection Questions

- Why run two passes with different prompts instead of one pass, or the same prompt twice?
- A line flagged by only one pass becomes “contested” rather than auto-cleared or dropped. Why surface disagreement to a human instead of resolving it automatically?
- Confirmed findings still pass through the confidence threshold. Why combine agreement AND confidence rather than trusting agreement alone?

5. Debrief & Reflection

5.1 Self-Check Before You Leave

Answer each question without looking back.

#	Question
1	How does a confidence score route findings, and what does the threshold trade off?
2	Why is the quote attached locally from the report rather than returned by the model?
3	Why is a JSON parse failure treated as zero findings, and why drop out-of-range lines?
4	How are confirmed and contested findings decided, and why two different prompts?
5	Why combine agreement AND confidence before auto-clearing a finding?

5.2 Common Mistakes

Mistake	Why it matters and what to do instead
Auto-clearing everything to save time.	A wrong auto-clear is a missed compliance issue. Score confidence and route the low end to a human; tune the threshold deliberately.

Mistake	Why it matters and what to do instead
Letting the model return the quote text.	A model-copied quote can be altered or invented. Attach the quote locally from the source by line number so it can't be hallucinated.
Crashing on a malformed model reply.	One bad response shouldn't take down the run. Parse with <code>json.loads</code> after stripping fences; treat failure as a low-confidence skip.
Trusting a single review pass.	One opinion isn't corroboration. Run two passes with different prompts; confirm on agreement and surface disagreement as contested.
Auto-clearing on agreement alone.	Two passes can agree and both be unsure. Require agreement AND confidence above threshold before clearing without human eyes.

6. Quick Reference

6.1 The Finding & the Three Buckets

```
finding = {"line": 4, "quote": "<exact source line>",
          "flag": "unusual_change_without_disclosure", "confidence": 0.92}

auto_clear    -> in BOTH passes AND avg confidence >= CONFIDENCE_THRESHOLD (0.75)
human_review  -> in both passes BUT below the threshold
contested     -> flagged by only ONE of the two passes
```

6.2 Confidence Routing

```
CONFIDENCE_THRESHOLD = 0.75
avg = (a["confidence"] + b["confidence"]) / 2
auto_clear.append(entry) if avg >= CONFIDENCE_THRESHOLD else
human_review.append(entry)
# raise the threshold -> safer, more human review; lower it -> faster, more auto-clear
```

6.3 Provenance (attach locally, never trust the model's copy)

```
quote = get_quote(f["line"])      # from OUR report data, by line number
if not quote: continue           # out of range -> drop (unverifiable)
# parser: strip ``` fences; on json.JSONDecodeError return [] (safe skip)
```

6.4 Confirmed vs. Contested (two passes)

```
pass_a = review(report, mode="strict")    # two different prompts =
pass_b = review(report, mode="general")    # two semi-independent reviewers
# same line in both -> confirmed (route by confidence)
# line in only one -> contested (show both passes' flags; send to a human)
```

6.5 File Inventory

File	Role	Purpose
<code>main.py</code>	entry	Runs two passes and prints the three buckets with provenance.
<code>confidence.py</code>	D 1/3 (S5)	Buckets findings into auto_clear / human_review / contested.
<code>reviewer.py</code>	D 2 (S6)	Claude review pass; parses JSON and attaches the quote locally.
<code>sample_report.py</code>	data	Mock report (one line each); <code>get_numbered_report()</code> & <code>get_quote()</code> .
<code>.env.example</code> , <code>requirements.txt</code>	setup	API key template; anthropic + python-dotenv dependencies.

6.6 Further Reading

- Anthropic API overview: <https://docs.claude.com/en/api/overview>
- Structured outputs / tool use: <https://docs.claude.com/en/docs/build-with-claude/tool-use>
- Evaluation, confidence & provenance patterns: <https://docs.claude.com/en/docs/build-with-claude>
- Optional background (no dedicated CCA-F lesson for S5/S6): AI Capabilities and Limitations